

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

print("Libraries Imported Successfully")
```

Libraries Imported Successfully

```
import pandas as pd
import numpy as np

np.random.seed(42)

n = 1000

tv = np.random.uniform(0, 300, n)
radio = np.random.uniform(0, 50, n)
newspaper = np.random.uniform(0, 120, n)

noise = np.random.normal(0, 2, n)

sales = (
    3
    + 0.05 * tv
    + 0.18 * radio
    + 0.01 * newspaper
    + noise
)

sales = np.round(sales, 2)

df = pd.DataFrame({
    "TV": np.round(tv, 2),
    "Radio": np.round(radio, 2),
    "Newspaper": np.round(newspaper, 2),
    "Sales": sales
})

df.to_csv("Advertising.csv", index=False)

print("Advertising.csv created successfully!")
print("Shape:", df.shape)

df.head()
```

Advertising.csv created successfully!

Shape: (1000, 4)

	TV	Radio	Newspaper	Sales
0	112.36	9.26	31.40	12.72
1	285.21	27.10	29.64	23.67
2	219.60	43.65	108.75	24.29
3	179.60	36.61	29.95	16.14
4	46.81	40.33	32.63	15.35



Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df = pd.read_csv("Advertising.csv")
```

```
df.head()
```

	TV	Radio	Newspaper	Sales
0	112.36	9.26	31.40	12.72
1	285.21	27.10	29.64	23.67
2	219.60	43.65	108.75	24.29
3	179.60	36.61	29.95	16.14
4	46.81	40.33	32.63	15.35



Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
print("Shape of Dataset:", df.shape)
```

```
df.info()
```

```
df.describe()
```

```

Shape of Dataset: (1000, 4)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   TV           1000 non-null   float64
1   Radio        1000 non-null   float64
2   Newspaper    1000 non-null   float64
3   Sales        1000 non-null   float64
dtypes: float64(4)
memory usage: 31.4 KB

```

	TV	Radio	Newspaper	Sales
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	147.076940	25.35080	60.288610	15.52452
std	87.641272	14.60942	34.880997	5.48793
min	1.390000	0.16000	0.000000	0.86000
25%	70.792500	12.05250	31.357500	11.53000
50%	149.040000	25.93500	60.070000	15.51500
75%	223.292500	38.02000	91.092500	19.43000
max	299.920000	49.97000	119.740000	30.84000

```
print(df.isnull().sum())
```

```

TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64

```

```
X = df[['TV', 'Radio', 'Newspaper']]
```

```
y = df['Sales']
```

```

print("Features Shape:", X.shape)
print("Target Shape:", y.shape)

```

```

Features Shape: (1000, 3)
Target Shape: (1000,)

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

```

```
print("Training Samples:", len(X_train))  
print("Testing Samples:", len(X_test))
```

```
Training Samples: 800  
Testing Samples: 200
```

```
model = LinearRegression()  
  
model.fit(X_train, y_train)  
  
print("Model Trained Successfully")
```

```
Model Trained Successfully
```

```
coefficients = pd.DataFrame({  
    'Feature': X.columns,  
    'Coefficient': model.coef_  
})  
  
print(coefficients)  
  
print("\nIntercept:", model.intercept_)
```

	Feature	Coefficient
0	TV	0.049315
1	Radio	0.169423
2	Newspaper	0.010335

```
Intercept: 3.378192693858482
```

```
y_pred = model.predict(X_test)  
  
print("Predictions Generated")
```

```
Predictions Generated
```

```
comparison = pd.DataFrame({  
    'Actual': y_test,  
    'Predicted': y_pred  
})  
  
comparison.head(10)
```

	Actual	Predicted	
521	18.20	17.095542	
737	20.09	17.783117	
740	14.19	17.687311	
660	14.13	14.319949	
411	16.82	18.557035	
678	18.22	19.329801	
626	14.38	12.034511	
513	18.13	16.978371	
859	11.65	10.737913	
136	17.76	18.362688	

Next steps:

[Generate code with comparison](#)

[New interactive sheet](#)

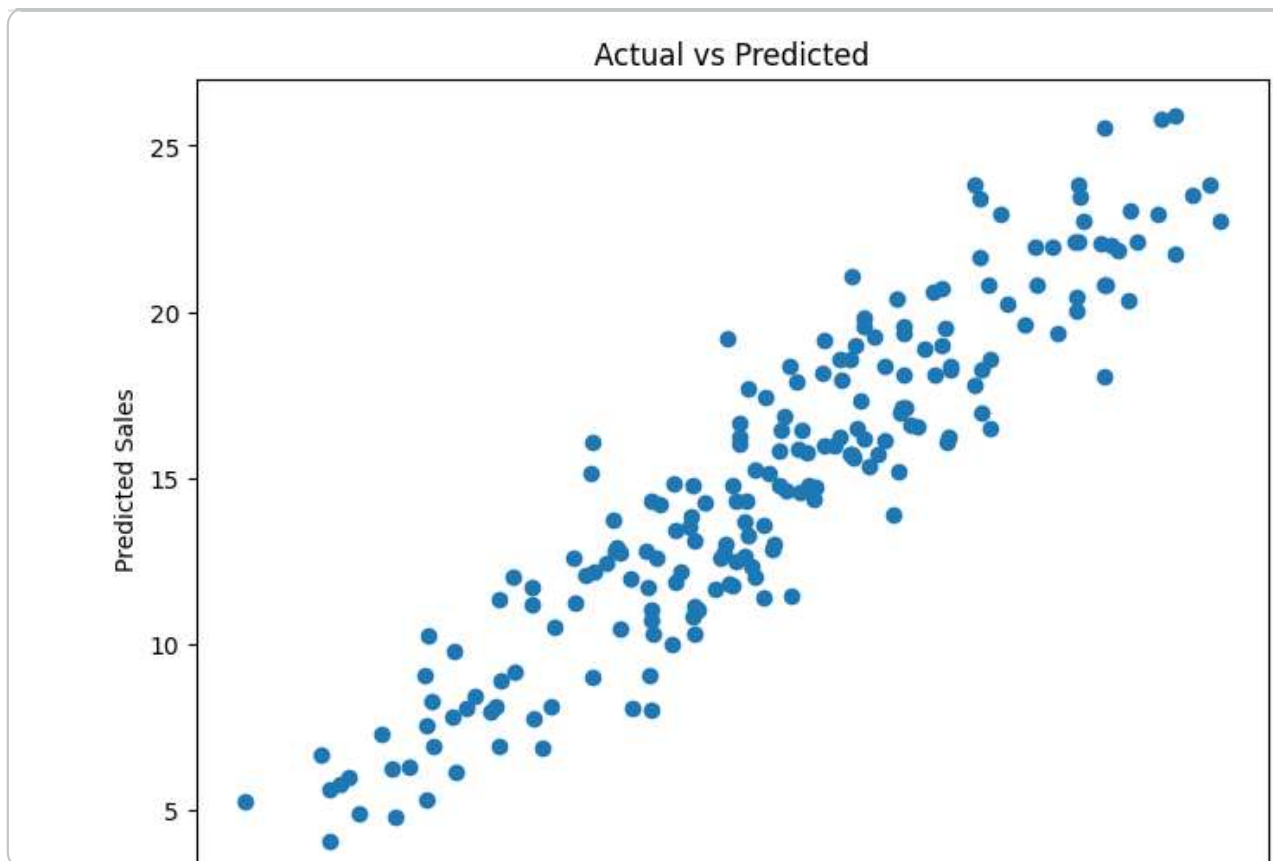
```
plt.figure(figsize=(8,6))

plt.scatter(y_test, y_pred)

plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")

plt.title("Actual vs Predicted")

plt.show()
```



```
mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
MAE : 1.7558354095043252
MSE : 4.440779490791165
RMSE: 2.107315707432364
R2 Score: 0.8589038418568691
```

```
residuals = y_test - y_pred

plt.figure(figsize=(8,6))

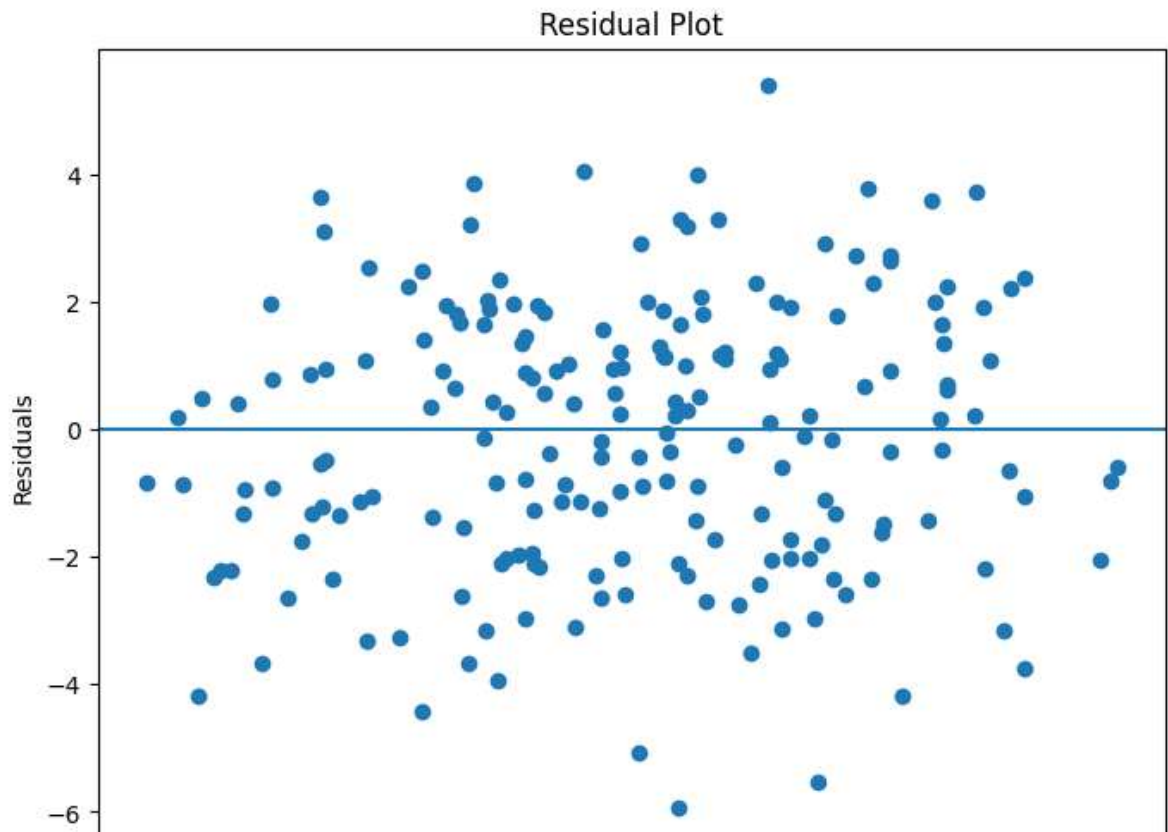
plt.scatter(y_pred, residuals)

plt.axhline(y=0)

plt.xlabel("Predicted Values")
plt.ylabel("Residuals")

plt.title("Residual Plot")
```

```
plt.show()
```



```
plt.figure(figsize=(8,5))
```